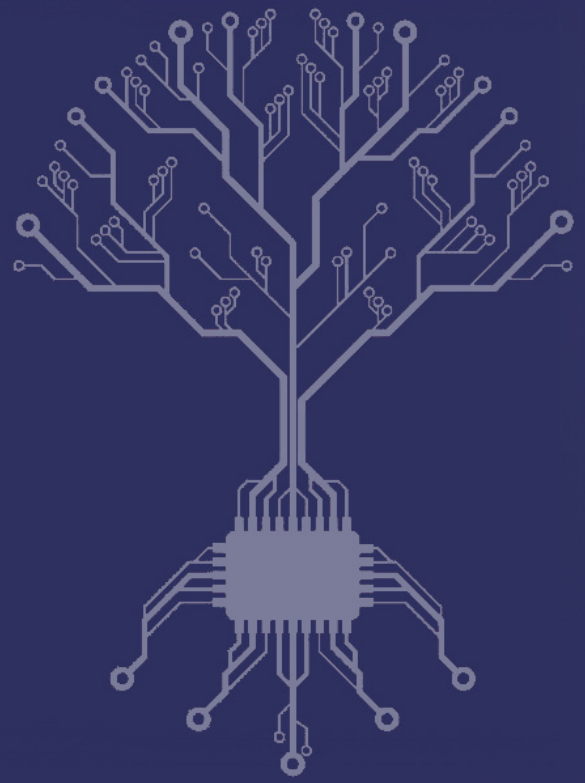
الأقسام التحضيرية للمدارس العليا

المملكة المغربية

وزارة التربية الوطنية والتعليم الأولي والرياضة

Base de Données (3/3)

Interrogation (Suite)



Opérations ensemblistes

Soit R_1 et R_2 deux relations ayant les mêmes attributs :

- *Union* $R_1 \cup R_2$: les lignes présentes dans R_1 ou dans R_2 ;
- *Intersection* $R_1 \cap R_2$: les lignes présentes à la fois dans R_1 et dans R_2 ;
- *Différence* $R_1 - R_2$: les lignes présentes dans R_1 mais pas dans R_2.

Syntaxe

SELECT ... FROM ... UNION SELECT ... FROM ... ;

SELECT ... FROM ... INTERSECT SELECT ... FROM ... ;

SELECT ... FROM ... EXCEPT SELECT ... FROM ... ;

(les deux requêtes renvoient le même nombre de colonnes ; les doublons sont supprimés)

Exemples

Soit les relations :

student(*student_id*, *first_name*, *last_name*, *field*, *average*, *city*, #*center_code*)
center(*center_code*, *name*, *city*)

- Villes des étudiants et des centres :
  $\pi_{city}(student) \cup \pi_{city}(center)$
  - SELECT city FROM student UNION SELECT city FROM center ;
- Centres ayant des étudiants des deux filières :
  $\pi_{center_code}(\sigma_{field='Mathematics'}(student)) \cap \pi_{center_code}(\sigma_{field='Economics'}(student))$
  - SELECT center_code FROM student WHERE field = *'Mathematics'*
    INTERSECT
    SELECT center_code FROM student WHERE field = *'Economics'* ;
- Centres qui n'accueillent aucun étudiant :
  $\pi_{center_code}(center) - \pi_{center_code}(student)$
  - SELECT center_code FROM center EXCEPT SELECT center_code FROM student ;

Produit cartésien

Le *produit cartésien* de deux relations R_1 et R_2, noté $R_1 \times R_2$, est la relation formée de toutes les combinaisons d'une ligne de R_1 avec une ligne de R_2.

Si R_1 a n_1 lignes et R_2 a n_2 lignes, $R_1 \times R_2$ a $n_1 \times n_2$ lignes.

Syntaxe

SELECT ... FROM Table1, Table2 ;

SELECT ... FROM Table1 CROSS JOIN Table2 ;

Exemple

student_id	*first_name*	*last_name*	*field*	*average*	*city*	*center_code*
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301
4	Laila	Ahmad	Economics	14.4	Rabat	CP102

$\times$

center_code	*name*	*city*
CP102	Omar Al Khayyam	Rabat
CP301	Mohammed VI	Kénitra

$=$

student_id	*first_name*	*last_name*	*field*	*average*	*student. city*	*student. center_code*	*center. center_code*	*name*	*center. city*
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102	CP102	Omar Al Khayyam	Rabat
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102	CP301	Mohammed VI	Kénitra
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301	CP102	Omar Al Khayyam	Rabat
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301	CP301	Mohammed VI	Kénitra
4	Laila	Ahmad	Economics	14.4	Rabat	CP102	CP102	Omar Al Khayyam	Rabat
4	Laila	Ahmad	Economics	14.4	Rabat	CP102	CP301	Mohammed VI	Kénitra

$3 \times 2 = 6$ lignes. Seules les lignes en blanc associent un étudiant à son propre centre (*student.center_code* $=$ *center.center_code*) : on les obtiendra directement avec la **jointure**.

Jointure

Joindre deux tables, qui sont en relation à l'aide d'une clé primaire et d'une clé étrangère, revient à créer une nouvelle table temporaire qui associe les lignes qui ont des valeurs égales dans la colonne commune des deux tables.

En algèbre relationnelle, c'est une sélection sur le produit cartésien :

$$R_1 \bowtie_{R_1.A_1 = R_2.A_2} R_2 = \sigma_{R_1.A_1 = R_2.A_2}(R_1 \times R_2)$$

Syntaxe

SELECT ... FROM Table1 JOIN Table2 ON Table1.colonne1=Table2.colonne2 ;

Ou bien

SELECT ... FROM Table1 JOIN Table2 USING (colonne) ;

(si la colonne commune porte le même nom dans les deux tables)

Exemple

Soit deux tables :

- *student*(*student_id*, *first_name*, *last_name*, *field*, *average*, *city*, #*center_code*)
- *center*(*center_code*, *name*, *city*)

student_id	*first_name*	*last_name*	*field*	*average*	*city*	*center_code*
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301
4	Laila	Ahmad	Economics	14.4	Rabat	CP102

$\bowtie$

center_code	*name*	*city*
CP102	Omar Al Khayyam	Rabat
CP301	Mohammed VI	Kénitra

$=$

student_id	*first_name*	*last_name*	*field*	*average*	*student.city*	*center_code*	*name*	*center.city*
2	Sarah	Mohammad	Economics	18.3	Rabat	CP102	Omar Al Khayyam	Rabat
3	Omar	Yussuf	Mathematics	16.0	Kénitra	CP301	Mohammed VI	Kénitra
4	Laila	Ahmad	Economics	14.4	Rabat	CP102	Omar Al Khayyam	Rabat

$$student \bowtie_{student.center_code = center.center_code} center$$

Exemple

- Pour afficher les moyennes des étudiants dont le centre est *'Mohammed VI'*, une jointure est nécessaire car la moyenne de l'étudiant et le nom du centre se trouvent dans des tables différentes :
  - SELECT average FROM student JOIN center
    ON student.center_code=center.center_code
    WHERE name = *'Mohammed VI'*;
  - ou SELECT average FROM student JOIN center USING (center_code)
    WHERE name = *'Mohammed VI'*;

La notation *table.colonne* désigne la colonne d'une table précise. Elle évite toute ambiguïté lorsque deux tables ont une colonne de même nom (par exemple *student.city* et *center.city*).

Requête imbriquée

Une requête imbriquée est une sous-requête exécutée à l'intérieur d'une autre requête SQL. Elle remplace souvent une constante au sein d'une clause WHERE ou HAVING.

Les sous-requêtes sont souvent utilisées avec l'instruction SELECT. Toutefois, on peut également les utiliser dans une instruction INSERT, UPDATE, DELETE ou dans une autre sous-requête.

Syntaxe

SELECT ... FROM ... WHERE ... opérateur (SELECT ... FROM ...) ;

Opérateur peut être :

- Si la requête renvoie un seul résultat :
  - Un opérateur simple : =, <, >, ! =, <=, >=
- Si elle renvoie plusieurs résultats :
  - IN
    NOT IN
  - EXISTS (la ligne est gardée si la sous-requête renvoie au moins une ligne)
    NOT EXISTS (la ligne est gardée si la sous-requête ne renvoie rien)

    La sous-requête utilise en général les valeurs de la ligne en cours de la requête principale.

Exemple 1

- Afficher les étudiants dont la note est supérieure ou égale à la moyenne des notes de tous les étudiants :
  - SELECT * FROM student WHERE average >= (SELECT AVG(average) FROM student) ;

- La sous-requête est exécutée une seule fois et renvoie une seule valeur : la moyenne générale.
- La requête principale compare ensuite la note de chaque étudiant à cette valeur et garde ceux dont la note est supérieure ou égale.
- On ne peut pas écrire directement WHERE average >= AVG(average) : une fonction d'agrégation n'est pas autorisée dans WHERE.

Exemple 2

- Afficher l'étudiant qui a obtenu la meilleure note :
  - SELECT * FROM student WHERE average = (SELECT MAX(average) FROM student) ;

- La sous-requête renvoie la meilleure note.
- La requête principale garde l'étudiant dont la note est égale à cette valeur.
- SELECT MAX(average) FROM student seule donne la note, mais pas l'étudiant qui l'a obtenue : d'où la sous-requête.
- Si plusieurs étudiants ont la même meilleure note, ils sont tous affichés.

Exemple 3

- Soit les tables :

  student(*student_id*, *first_name*, *last_name*, *field*, *average*, *city*, #*center_code*)
  book(*ISBN*, *title*, *author*)
  borrow(#*student_id*, #*ISBN*, *date*)

- Afficher les étudiants qui ont emprunté un livre.

Exemple 3 (avec IN)

- SELECT student_id, first_name, last_name FROM student WHERE student_id IN (SELECT DISTINCT student_id FROM borrow) ;

- La sous-requête est exécutée une seule fois et renvoie la liste, sans doublons, des *student_id* présents dans la table *borrow*.
- La requête principale garde les étudiants dont le *student_id* appartient (IN) à cette liste.

Exemple 3 (avec EXISTS)

- SELECT student_id, first_name, last_name FROM student WHERE EXISTS (SELECT * FROM borrow WHERE student.student_id=borrow.student_id) ;

- Pour chaque étudiant de la requête principale, la sous-requête cherche dans *borrow* les emprunts ayant le même *student_id*.
- L'étudiant est gardé si la sous-requête renvoie au moins une ligne.